

Name: Muhammad Asim

Roll NO:SU92-BSSEM-F25-054

Section: 3A-BSSE Semester: 3rd

Professor: Muhammad Asad

Task NO: 03

Q NO: 01

Design a C++ program to create a class named Product. The class should contain the following private data members:

- productID — to store the product ID
- productName — to store the product name
- price — to store the product price

Create public member functions named setData() using function overloading:

- One setData() function should take the product ID and product name as arguments.
- Another overloaded setData() function should take the product ID, product name, and price as arguments.

In the main() function:

1. Ask the user how many products they want to enter.
2. Use a for loop to repeatedly take data for each product.
3. Create an object of the Product class for each product.
4. Use the appropriate overloaded setData() function to store the entered information.

5. For some products, enter only the ID and name, while for others, enter the ID, name, and price.

6. Demonstrate how function overloading allows the same function name setData() to perform different tasks based on the number of arguments.

```
#include <iostream>

using namespace std;


class Product

{

private:

   int productID;

   string productName;

   double price;


public:


   void setData(int id, string name)

   {

      productID = id;

      productName = name;

      price = 0;

   }


   void setData(int id, string name, double p)

   {

      productID = id;

      productName = name;
```

```
        price = p;
    }

    void display()
    {
        cout << "Product ID: " << productID << endl;
        cout << "Product Name: " << productName << endl;
        cout << "Price: " << price << endl;
    }
};

int main()
{
    int n;

    cout << "Enter number of products: ";
    cin >> n;

    for (int i = 0; i < n; i++)
    {
        Product p;

        int id;
        string name;
        double price;
        char choice;
```

```
    cout << "\nProduct " << i + 1 << endl;

    cout << "Enter Product ID: ";
    cin >> id;

    cout << "Enter Product Name: ";
    cin >> name;

    cout << "Do you want to enter price? (Y/N): ";
    cin >> choice;

    if (choice == 'Y' || choice == 'y')
    {
        cout << "Enter Price: ";
        cin >> price;
        p.setData(id, name, price);
    }
    else
    {
        p.setData(id, name);
    }

    cout << "\nProduct Information:" << endl;
    p.display();
}
```

```
    return 0;
}
```

Q NO: 02

Write a C++ program to develop a simple Bank Account Management System using Object-Oriented Programming concepts. Create a class named BankAccount to store and manage basic information about a bank account.

The class should contain the following private data members:

- accountNumber — to store the account number
- accountHolder — to store the account holder's name
- accountType — to store the type of account (e.g., Savings or Current)
- balance — to store the current account balance

The class must include the following requirements:

1. Create a default constructor that initializes the account with default values such as account number 0, account holder "Unknown", account type "Savings", and balance 0.

2. Create a parameterized constructor that accepts the account number, account holder name, account type, and initial balance as arguments and initializes the data members.

3. Implement constructor overloading by creating another constructor that accepts only the account number and account holder name. In this case, the account type should automatically be set to "Savings" and the balance should be initialized to 0.

4. Create a member function named deposit() that accepts an amount and adds it to the account balance. The function should display an appropriate message after a successful deposit.

5. Create another member function named withdraw() that accepts an amount and subtracts it from the balance only if sufficient funds are available. If the balance is insufficient, display an appropriate error message.

6. Create a member function named displayAccount() to display all the details of the bank account, including account number, account holder, account type, and current balance.

7. Create a destructor for the class. The destructor should display a message indicating that the particular bank account object is being destroyed.

8. In the main() function, create three different objects of the BankAccount class:

o The first object should use the default constructor.

o The second object should use the constructor with account number and account holder name.

o The third object should use the fully parameterized constructor.

9. Display the information of all three accounts using displayAccount().

10. Perform at least one deposit and one withdrawal operation on the third account and display the updated balance.

11. Use appropriate messages to clearly demonstrate which constructor is called for each object.

12. At the end of the program, observe and demonstrate the order in which the destructors are called when the objects go out of scope.

```
#include <iostream>

using namespace std;


class BankAccount

{

private:

   int accountNumber;

   string accountHolder;

   string accountType;

   double balance;


public:

   BankAccount()

   {

      accountNumber = 0;
```

```
    accountHolder = "Unknown";
    accountType = "Savings";
    balance = 0;

    cout << "Default Constructor Called" << endl;
}

BankAccount(int number, string holder)
{
    accountNumber = number;
    accountHolder = holder;
    accountType = "Savings";
    balance = 0;

    cout << "2-Argument Constructor Called" << endl;
}

BankAccount(int number, string holder, string type, double amount)
{
    accountNumber = number;
    accountHolder = holder;
    accountType = type;
    balance = amount;

    cout << "4-Argument Constructor Called" << endl;
}
```

```
void deposit(double amount)
{
   balance = balance + amount;

   cout << "Deposit Successful!" << endl;
   cout << "Deposited Amount: " << amount << endl;
}

void withdraw(double amount)
{
   if (amount <= balance)
   {
      balance = balance - amount;

      cout << "Withdrawal Successful!" << endl;
      cout << "Withdrawn Amount: " << amount << endl;
   }
   else
   {
      cout << "Error: Insufficient Balance!" << endl;
   }
}
void displayAccount()
{
   cout << "\nAccount Number: " << accountNumber << endl;
   cout << "Account Holder: " << accountHolder << endl;
```

```
        cout << "Account Type: " << accountType << endl;
        cout << "Balance: " << balance << endl;
    }
    ~BankAccount()
    {
        cout << "Destructor Called for Account: "
             << accountNumber << endl;
    }
};

int main()
{
    cout << "Creating Account 1..." << endl;

    BankAccount account1;

    cout << "\nCreating Account 2..." << endl;

    BankAccount account2(102, "Ali");

    cout << "\nCreating Account 3..." << endl;

    BankAccount account3(103, "Ahmed", "Current", 50000);
    cout << "\n===== Account 1 =====";
    account1.displayAccount();
```

```
    cout << "\n===== Account 2 =====";
    account2.displayAccount();

    cout << "\n===== Account 3 =====";
    account3.displayAccount();

    cout << "\n===== Deposit Operation =====" << endl;
    account3.deposit(10000);

    cout << "\n===== Withdrawal Operation =====" << endl;
    account3.withdraw(5000);
    cout << "\n===== Updated Account 3 =====";
    account3.displayAccount();

    cout << "\nProgram Ending..." << endl;

    return 0;
}
```